p.1 해설 — 초록: 어텐션만으로 충분하다

① **맨 위 안내문** — 구글은 출처를 밝히면 이 논문의 **표와 그림을 언론·학술 목적으로 복제**하는 것을 허락한다고 적혀 있다.

② **초록** — 기존 시퀀스 변환(번역 등) 모델은 **인코더-디코더** 구조의 복잡한 RNN 또는 CNN이고, 최고 모델들은 둘을 **어텐션**으로 연결한다. 이 논문은 **순환과 합성곱을 완전히 버리고** 어텐션만으로 만든 단순한 구조 **트랜스포머**를 제안.

- 품질은 더 좋고, **병렬화**가 쉬워 학습 시간이 훨씬 짧다
- WMT 2014 영→독: **BLEU 28.4** (앙상블 포함 기존 최고보다 2 이상 향상)
- WMT 2014 영→불: 단일 모델 최고 **BLEU 41.8**, GPU 8개로 3.5일
- 영어 구문 분석에도 잘 일반화 → 🔎 *dispensing with recurrence and convolutions entirely* 에 밑줄.

③ **각주: 저자 기여** — 저자 순서는 **무작위**. Jakob: RNN을 셀프 어텐션으로 대체하자는 제안. Ashish·Illia: 첫 트랜스포머 설계·구현. **Noam: 스케일드 닷-프로덕트 어텐션, 멀티-헤드 어텐션, 파라미터 없는 위치 표현** 제안. Niki: 수많은 변형 실험. Llion: 초기 코드와 시각화. Lukasz·Aidan: tensor2tensor 구현. → 한 논문의 아이디어가 어떻게 팀으로 나뉘어 만들어지는지 보여주는 드문 각주.

④ **학회** — NIPS 2017 (현 NeurIPS), 롱비치. 이 PDF는 arXiv v7 (2023년).

p.2 해설 — 1절 서론, 2절 배경, 3절 시작

① **RNN의 한계** ⭐ — RNN, LSTM, GRU가 언어 모델링·번역의 표준. RNN은 위치 t의 은닉 상태 h_t를 h_{t-1}과 입력으로 계산 → **본질적으로 순차적** → 한 예제 안에서 병렬화 불가. 문장이 길면 메모리 제약으로 더 문제. → 🔎 *This inherently sequential nature precludes parallelization* 에 밑줄.

② **어텐션** — 입력·출력에서 **거리에 상관없이** 의존성을 모델링. 하지만 대부분 RNN과 함께 쓰임.

③ **트랜스포머의 제안** — 순환을 버리고 어텐션만으로 입력-출력의 **전역 의존성**을 포착. P100 GPU 8개로 **12시간** 학습만에 번역 최고 성능.

④ **2절 배경: CNN 기반 시도** — Extended Neural GPU, ByteNet, ConvS2S는 CNN으로 병렬 계산. 하지만 두 위치를 연결하는 연산 수가 거리에 따라 증가 (ConvS2S는 선형, ByteNet은 로그) → 먼 의존성 학습이 어렵다. 트랜스포머는 이를 **상수**로 줄인다 (대신 평균화로 해상도가 떨어지는데, 이를 **멀티-헤드**로 보완).

⑤ **셀프 어텐션** — **한 시퀀스 안의** 서로 다른 위치를 연결해 그 시퀀스의 표현을 계산. 독해, 요약, 함의 등에서 이미 성공.

⑥ **3절 모델 구조 시작** — 인코더: 입력 $(x_1, \ldots, x_n)$ → 연속 표현 $z = (z_1, \ldots, z_n)$. 디코더: z로부터 출력 $(y_1, \ldots, y_m)$ 을 **한 번에 하나씩** 생성. 매 단계 이전 출력을 입력으로 쓰는 **자기회귀(auto-regressive)**.

p.3 해설 — Figure 1: 트랜스포머 구조 ⭐

---

① **Figure 1⭐ 이 그림을 손으로 그려 보세요** — 왼쪽 = 인코더, 오른쪽 = 디코더.

- 아래: 입력 임베딩 + **위치 인코딩**
- 인코더 블록 ×N: 멀티-헤드 어텐션 → Add & Norm → 피드포워드 → Add & Norm
- 디코더 블록 ×N: **마스크드** 멀티-헤드 어텐션 → Add & Norm → (인코더 출력과의) 멀티-헤드 어텐션 → Add & Norm → 피드포워드 → Add & Norm
- 위: 선형 → 소프트맥스 → 출력 확률

② **3.1 인코더** — 동일한 층 $N = 6$개. 각 층에 하위층 2개: (1) 멀티-헤드 셀프 어텐션, (2) 위치별 완전연결 피드포워드. 각 하위층에 **잔차 연결** + **층 정규화**:

$$\mathrm{LayerNorm}\big(x + \mathrm{Sublayer}(x)\big)$$

잔차 연결을 위해 모든 출력 차원은 $d_{\text{model}} = 512$. → 🔎 잔차 연결은 이번 주 **ResNet 논문**의 $F(x) + x$ 그대로! 참고문헌 [11]이 바로 He 외.

③ **디코더** — 역시 $N = 6$. 세 번째 하위층: 인코더 출력에 대한 멀티-헤드 어텐션. 디코더의 셀프 어텐션은 **뒤쪽 위치를 보지 못하도록 마스킹** + 출력 임베딩을 한 칸 밀기 → 위치 i의 예측은 i 미만의 출력에만 의존. → "미래를 커닝하지 못하게" 하는 장치. GPT 같은 생성 모델의 핵심.

④ **3.2 어텐션의 정의** — 어텐션 = **질의(query)** 와 **키-값(key-value) 쌍**들을 출력으로 사상하는 함수. 출력 = 값들의 **가중합**, 가중치 = 질의와 각 키의 **호환성 함수**.

p.4 해설 — 스케일드 닷-프로덕트 어텐션 ⭐ 가장 중요한 쪽

① **Figure 2** — 왼쪽: 스케일드 닷-프로덕트 어텐션 (MatMul → Scale → Mask(선택) → SoftMax → MatMul). 오른쪽: 여러 어텐션을 **병렬**로 돌리는 멀티-헤드.

② **3.2.1 식 (1) ⭐ 반드시 손으로 계산해 보기**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

질의·키 차원 d_k, 값 차원 d_v. 질의와 모든 키의 내적 → $\sqrt{d_k}$로 나눔 → 소프트맥스 → 값의 가중치. → ✍️ Q, K, V를 3×2 행렬로 잡고 직접 계산해 보세요.

③ **두 종류의 어텐션**

- **덧셈형**: 은닉층 하나짜리 피드포워드로 호환성 계산
- **곱셈형(내적)**: 이 논문 방식에서 스케일만 뺀 것. 고도로 최적화된 **행렬곱**으로 구현 가능 → 훨씬 빠르고 메모리 효율적

④ **왜 $\sqrt{d_k}$로 나누나? ⭐ 확률 계산** — d_k가 크면 스케일 없는 내적은 덧셈형보다 나쁘다. 내적 값이 커져 소프트맥스가 **기울기가 극히 작은 영역**으로 밀려나기 때문. **각주 4**: q, k의 성분이 평균 0, 분산 1인 독립 확률변수이면

$$\mathbb{E}[q \cdot k] = 0, \qquad \text{Var}(q \cdot k) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$$

→ $\sqrt{d_k}$로 나누면 분산이 1로 맞춰진다. → 🔎 각주 4에 크게 표시. 이 논문에서 가장 "수학적인" 한 줄.

⑤ **3.2.2 멀티-헤드 시작** — d_{model} 차원 하나로 어텐션 하는 대신, Q, K, V를 h**번 서로 다른 학습된 선형 사영**으로 d_k, d_k, d_v 차원에 보내 병렬로 어텐션.

p.5 해설 — 멀티-헤드, 세 가지 어텐션, FFN, 임베딩

① 멀티-헤드 어텐션

$$\mathrm{MultiHead}(Q,K,V)=\mathrm{Concat}(\mathrm{head}_1,\dots,\mathrm{head}_h)W^O,\quad \mathrm{head}_i=\mathrm{Attention}(QW_i^Q,KW_i^I$$

서로 다른 **표현 부분공간**의 정보를 동시에 볼 수 있음. 헤드가 하나면 평균화가 이를 방해. 설정: $h=8$, $d_k=d_v=d_{\text{model}}/h=64$ → 전체 계산량은 단일 헤드와 비슷.

② 3.2.3 어텐션의 세 가지 쓰임 ⭐

종류	질의 출처	키·값 출처
인코더-디코더 어텐션	이전 디코더 층	**인코더 출력**
인코더 셀프 어텐션	이전 인코더 층	이전 인코더 층
디코더 셀프 어텐션	이전 디코더 층	이전 디코더 층 (**자기 위치까지만**)
마스킹 방법: 소프트맥스 입력에서 금지된 연결을 $-\infty$ 로 설정 → $e^{-\infty}=0$.		

③ 3.3 위치별 피드포워드

$$\mathrm{FFN}(x)=\max(0,\ xW_1+b_1)W_2+b_2 \tag{2}$$

각 위치에 **따로, 동일하게** 적용. = 커널 크기 1인 합성곱 두 개. $d_{\text{model}}=512$, 내부 차원 $d_{ff}=2048$.

④ 3.4 임베딩과 소프트맥스 — 학습된 임베딩으로 토큰 → d_{model} 차원 벡터. 두 임베딩 층과 소프트맥스 직전 선형층이 **같은 가중치 행렬을 공유**. 임베딩에는 $\sqrt{d_{\text{model}}}$을 곱함.

p.6 해설 — Table 1, 위치 인코딩, 4절 왜 셀프 어텐션인가

① **Table 1** ⭐ — n = 시퀀스 길이, d = 표현 차원, k = 합성곱 커널, r = 제한된 이웃 크기.

층	층당 복잡도	순차 연산	최대 경로 길이
셀프 어텐션	$O(n^2 d)$	$O(1)$	$O(1)$
순환(RNN)	$O(nd^2)$	$O(n)$	$O(n)$
합성곱	$O(knd^2)$	$O(1)$	$O(\log_k n)$
제한된 셀프 어텐션	$O(rnd)$	$O(1)$	$O(n/r)$

② **3.5 위치 인코딩** ⭐ — 순환도 합성곱도 없으므로 **순서 정보를 주입**해야 한다. 임베딩에 같은 차원의 위치 인코딩을 **더한다**:

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{\text{model}}}\right), \qquad PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{\text{model}}}\right)$$

파장은 2π부터 $10000 \cdot 2\pi$까지 **등비수열**. → 🔎 *for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_pos* 에 밑줄. → ✍️ 덧셈정리로 증명해 보세요: $\sin(a+b) = \sin a \cos b + \cos a \sin b$
→ $(\sin, \cos)$ 쌍에 **회전 행렬**을 곱한 것.

③ **학습된 위치 임베딩과 비교** — 거의 같은 결과 (Table 3 (E)). 사인 버전을 고른 이유: 학습 때보다 **긴 시퀀스로 외삽**할 수 있을지도 몰라서.

④ **4절: 세 가지 기준** — (1) 층당 총 계산 복잡도, (2) 병렬화 가능한 양 (최소 순차 연산 수), (3) 장거리 의존성 사이의 **경로 길이** — 짧을수록 학습이 쉽다.

p.7 해설 — 셀프 어텐션의 장점, 5절 학습

① **복잡도 비교** — 셀프 어텐션은 $n < d$ 일 때 RNN보다 빠르다. 번역 모델의 문장 표현(단어 조각, BPE)은 대부분 이 경우. 아주 긴 시퀀스는 이웃 r로 **제한**하면 되지만 경로 길이가 $O(n/r)$로 늘어남. → 오늘날 거대 언어모델의 "긴 문맥" 문제가 바로 이 $O(n^2)$ 에서 나온다.

② **합성곱과 비교** — 커널 폭 $k < n$ 인 합성곱 층 하나로는 모든 위치 쌍이 연결되지 않음 → $O(n/k)$ 층(연속 커널) 또는 $O(\log_k n)$ 층(팽창 합성곱) 필요. 분리 가능 합성곱도 $k = n$ 이면 셀프 어텐션 + 피드포워드와 같은 복잡도.

③ **해석 가능성** — 어텐션 분포를 들여다보면 헤드마다 **다른 일**을 하고, 많은 헤드가 문장의 **문법·의미 구조**와 관련된 행동을 보인다 (부록 그림).

④ **5절 학습 데이터** — WMT 2014 영-독 약 **450만** 문장 쌍, 바이트 쌍 인코딩(BPE) 공유 어휘 약 37,000 토큰. 영-불 **3,600만** 문장, 단어 조각 32,000. 배치당 원문·번역문 각 약 25,000 토큰.

⑤ **하드웨어** — NVIDIA P100 GPU 8개. 기본 모델: 스텝당 0.4초, 10만 스텝 = **12시간**. 큰 모델: 스텝당 1.0초, 30만 스텝 = **3.5일**.

⑥ **최적화기** ⭐ — **Adam** ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$) — 이번 주 Adam 논문! 학습률 스케줄 (식 3):

$$lrate = d_{\text{model}}^{-0.5} \cdot \min\left(step^{-0.5},\ step \cdot warmup^{-1.5}\right), \quad warmup = 4000$$

처음 4000 스텝은 **선형 증가(워밍업)**, 이후 스텝의 **역제곱근**으로 감소. → ✍️ 두 항이 같아지는 지점이 $step = warmup$ 임을 확인해 보세요.

p.8 해설 — Table 2, 정규화, 6절 결과

① **Table 2** ⭐ — 번역 성능(BLEU)과 학습 비용(FLOPs).

모델	영→독	영→불	비용(영→독)
GNMT + RL	24.6	39.92	$2.3 \cdot 10^{19}$
ConvS2S	25.16	40.46	$9.6 \cdot 10^{18}$
ConvS2S 앙상블	26.36	41.29	$7.7 \cdot 10^{19}$
트랜스포머 (기본)	27.3	38.1	$\mathbf{3.3 \cdot 10^{18}}$
트랜스포머 (큰)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$
→ 🔎 기본 모델이 **훨씬 적은 비용**으로 기존 앙상블을 이긴다.			

② **5.4 정규화**

- **잔차 드롭아웃**: 각 하위층 출력에 드롭아웃 후 더하고 정규화. 임베딩 + 위치 인코딩 합에도. 기본 $P_{drop} = 0.1$.
- **레이블 평활화** $\epsilon_{ls} = 0.1$: 모델을 "덜 확신"하게 만들어 혼란도(perplexity)는 나빠지지만 **정확도와 BLEU는 개선**.

③ **6.1 번역 결과** — 영→독 큰 모델: 기존 최고(앙상블 포함)보다 **2.0 BLEU 이상** 높은 28.4. 영→불 큰 모델: 본문에는 **41.0** 으로 적혀 있음 (초록은 41.8 — 버전 간 불일치에 주의!), 이전 최고 모델 비용의 1/4 미만.

④ **추론 설정** — 기본 모델은 마지막 5개 체크포인트 **평균**, 큰 모델은 20개. 빔 크기 4, 길이 벌점 $\alpha = 0.6$. → 체크포인트 평균 = Adam 논문 7.2절의 "시간 평균"과 같은 아이디어!

⑤ **각주 5** — FLOPs 추정: 학습 시간 × GPU 수 × GPU당 지속 단정밀도 성능 (P100 = 9.5 TFLOPS).

p.9 해설 — Table 3: 구조 변형 실험, 6.3 구문 분석

① **Table 3 ⭐ 무엇이 중요한가를 실험으로** — 기본 모델: $N = 6$, $d_{\text{model}} = 512$, $d_{ff} = 2048$, $h = 8$, $d_k = d_v = 64$, $P_{drop} = 0.1$, $\epsilon_{ls} = 0.1$, 10만 스텝 → 개발셋 BLEU 25.8, 파라미터 6,500만.

② 변형별 결론

행	바꾼 것	결론
(A)	헤드 수 (계산량 고정)	헤드 1개는 0.9 BLEU 나쁨, **너무 많아도** 나빠짐
(B)	d_k 축소	품질 하락 → 호환성 판단이 쉽지 않다, 내적보다 정교한 함수가 나을 수도
(C)	모델 크기	**클수록 좋다**
(D)	드롭아웃	과적합 방지에 **매우 도움**
(E)	학습된 위치 임베딩	사인 인코딩과 거의 같음
big	$d_{\text{model}} = 1024$, $h = 16$, 30만 스텝	BLEU 26.4, 파라미터 2억 1,300만
→ 🔎 (C) "클수록 좋다" — 이번 주 Belkin 논문의 과대 파라미터 영역과 연결!		

③ **6.3 영어 구문 분석** — 다른 과제로 일반화되는지 확인. 출력이 강한 구조 제약을 받고 입력보다 훨씬 길다. RNN 모델은 작은 데이터에서 최고 성능을 못 냈다. 설정: 4층, $d_{\text{model}} = 1024$, WSJ 약 4만 문장 / 준지도 약 1,700만 문장.

p.10 해설 — Table 4, 7절 결론, 참고문헌 시작

① **Table 4: 구문 분석 결과 (F1)** — WSJ만: **91.3**, 준지도: **92.7**. 과제별 조정 없이도 RNN 문법(Recurrent Neural Network Grammar) 외의 모든 기존 모델보다 좋음. RNN 모델과 달리 4만 문장만으로도 BerkeleyParser를 이김.

② **7절 결론** ⭐

- 어텐션만으로 된 **최초의** 시퀀스 변환 모델
- 번역에서 RNN·CNN 기반보다 **훨씬 빠르게** 학습, 영-독·영-불 모두 최고 성능
- **미래 계획**: 텍스트 외의 입출력 — **이미지, 오디오, 비디오** — 로 확장, 큰 입력을 위한 국소·제한 어텐션, 생성을 덜 순차적으로 → 🔎 이 계획은 모두 실현되었다: ViT(이미지), Whisper(음성), 멀티모달 모델들. ai_life **02 컴퓨터비전**, **05 음성인식** 단원과 연결!

③ **코드 공개** — tensor2tensor (GitHub).

④ **참고문헌 시작**

번호	참고문헌	왜 볼까
[1]	Ba, Kiros, Hinton — 층 정규화	Add & Norm의 Norm
[2]	Bahdanau, Cho, Bengio (2014) ⭐	**어텐션의 원조** — 번역에서 정렬과 번역을 함께 학습

p.11 해설 — 참고문헌 [5]~[24]

참고문헌 중 주목할 것

번호	참고문헌	왜 볼까
[5]	Cho 외 (2014) RNN 인코더-디코더	인코더-디코더 구조의 시작, GRU
[7]	Chung 외 (2014)	게이트 순환 신경망 평가
[9]	Gehring 외 (2017) ConvS2S	CNN 기반 번역 — 비교 대상
[11]	**He 외 (2016) ResNet**	잔차 연결 — 이번 주 함께 읽는 논문
[12]	Hochreiter 외 (2001)	순환망의 장기 의존성 학습 어려움
[13]	Hochreiter & Schmidhuber (1997) LSTM	RNN 시대의 대표
[18]	Kalchbrenner 외 ByteNet	선형 시간 번역
[20]	**Kingma & Ba (2015) Adam**	이번 주 함께 읽는 논문
[24]	Luong 외 (2015)	어텐션 기반 번역의 효과적 접근

→ 🔎 한 논문의 참고문헌에 이번 주 논문이 두 편(ResNet, Adam)이나 등장합니다. 논문들이 서로 **쌓아 올려진** 구조를 확인하세요.

p.12 해설 — 참고문헌 [25]~[40]

참고문헌 중 주목할 것

번호	참고문헌	왜 볼까
[25]	Marcus 외 (1993) 펜 트리뱅크	구문 분석 데이터
[30]	Press & Wolf (2016)	출력 임베딩 공유 (p.5 가중치 공유)
[31]	Sennrich 외 (2015)	**바이트 쌍 인코딩(BPE)** — 오늘날 토크나이저의 기초
[32]	Shazeer 외 (2017) **전문가 혼합(MoE)**	거대 모델의 희소 구조 — 현대 LLM에도 쓰임
[33]	Srivastava 외 (2014) 드롭아웃	정규화
[35]	Sutskever, Vinyals, Le (2014) seq2seq	시퀀스-시퀀스 학습
[36]	Szegedy 외 (2015) Inception v3	**레이블 평활화**의 출처
[38]	Wu 외 (2016) 구글 신경망 번역(GNMT)	비교 대상, 단어 조각

✍️ 본문 정리 질문

1. Table 1의 세 기준(복잡도, 순차 연산, 경로 길이) 중 트랜스포머가 RNN을 이기는 결정적 이유는?
2. 셀프 어텐션의 $O(n^2)$ 는 어디서 오는가? ($QK^\top$ 의 크기)

p.13 해설 — 부록 Figure 3: 장거리 의존성

① **Figure 3** — 인코더 셀프 어텐션 **5번째 층**(6개 중)에서 단어 **'making'** 에 대한 어텐션. 색 = 서로 다른 헤드.

- 많은 헤드가 'making'의 **먼 의존성**에 주목하여 "making … more difficult" 구를 완성한다.

② **읽는 법** — 왼쪽 단어에서 오른쪽 단어로 이어진 선의 **진하기** = 어텐션 가중치(소프트맥스 출력)의 크기. → 🔎 'making'에서 멀리 떨어진 'more difficult'로 굵은 선이 가는지 확인하세요.

③ **의미** — RNN이라면 이 거리만큼 **순차적으로** 정보를 전달해야 하지만, 셀프 어텐션은 **한 번에** (경로 길이 $O(1)$) 연결한다. p.6 Table 1의 "최대 경로 길이" 열을 눈으로 확인하는 그림.

✍️ **생각해볼 것** — 이 시각화가 모델의 "이해"를 보여준다고 할 수 있을까? 어텐션 가중치를 **설명(해석)** 으로 쓰는 것의 한계도 생각해 보세요.

p.14 해설 — 부록 Figure 4: 대용어 해소

① **Figure 4** — 역시 5번째 층의 두 헤드(5번, 6번). **대용어 해소(anaphora resolution)** — 대명사가 무엇을 가리키는지 찾는 일 — 에 관여하는 것으로 보인다.

- **위**: 헤드 5의 전체 어텐션
- **아래**: 단어 **'its'** 에서 나가는 어텐션만 따로 (헤드 5, 6)

② **관찰** — 'its'의 어텐션이 **매우 날카롭다**(한두 단어에 집중). 'its'가 가리키는 명사(문장에서는 법/Law)에 집중.

③ **의미** — 아무도 "대명사를 해소하라"고 가르치지 않았는데, 번역을 잘하기 위해 모델이 스스로 이런 기능을 **학습**했다. → 🔎 이것이 "표현 학습(representation learning)"의 힘.

✍️ **ai_life 연결** — **04 자연어처리·AI 에이전트** 단원에서 학생들에게 "it/its가 무엇을 가리킬까?" 퀴즈를 내고, 이 그림과 비교해 보면 좋은 활동이 됩니다.

p.15 해설 — 부록 Figure 5: 문장 구조와 전체 정리

① **Figure 5** — 많은 어텐션 헤드가 **문장 구조**와 관련된 행동을 보인다. 인코더 5번째 층의 서로 다른 두 헤드 예시. 헤드들이 **명백히 다른 과제**를 수행하도록 학습되었다. → p.5 "멀티-헤드는 서로 다른 표현 부분공간의 정보를 동시에 본다"의 시각적 증거.

✍️ 논문 전체 정리 질문

1. $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$ 를 3×2 행렬로 직접 계산해 보자. (p.4)
2. 각주 4의 분산 계산 $\mathrm{Var}(q \cdot k) = d_k$ 를 증명해 보자.
3. 위치 인코딩에서 PE_{pos+k} 가 PE_{pos} 의 **회전**임을 덧셈정리로 보이자. (p.6)
4. 잔차 연결(ResNet)과 Adam이 트랜스포머에서 어떻게 쓰였는지 찾아 표시하자.
5. Table 3 (C) "클수록 좋다"를 Belkin의 이중 하강과 연결해 설명해 보자.
6. ai_life **04 텍스트 에이전트** 시뮬레이터에서 어텐션을 시각화한다면 어떻게 보여줄까?

핵심 용어	뜻
sequence transduction	시퀀스 변환
auto-regressive	자기회귀
label smoothing	레이블 평활화
anaphora resolution	대용어 해소
BLEU	번역 품질 점수